UVSim Milestone 4 design and use cases

Revised October 23, 2026. Current submission version; prior milestone files are historical.

Purpose and scope

UVSim is a Python-based BasicML simulator for students learning machine language and computer architecture. The main application uses a Tkinter graphical user interface (GUI) to open, view, edit, save, and execute BasicML programs.

Students can open a program from an accessible folder, review instructions and data at addresses 00–99, and add, modify, delete, cut, copy, or paste entries before running it. The editor validates signed four-digit words and prevents programs from exceeding the 100-word memory limit. Students can save the current program or use Save As to choose a different name or location.

During execution, the GUI accepts READ input, displays WRITE output and status messages, and provides controls to run or stop the program. The simulator supports all 12 required BasicML operations. A program can be run again without reopening its file. The default color scheme uses UVU green (#4C721D) and white (#FFFFFF), with colors configurable through a file.

UVSim requires Python 3.10 or newer and the Python standard library, including Tkinter/Tcl/Tk. No third-party packages are required. See `README.txt` for installation, controls, file format, and test instructions.

This document describes the integrated implementation. See verification.md for test evidence and remaining manual acceptance checks.

User stories

1. As a computer science student, I want to open and run a BasicML program so I can observe how instructions change memory and the accumulator.

2. As a student, I want to edit instructions and data in the GUI before execution so I can correct or experiment with my program.

3. As a student, I want to save my edits and choose where to store the program so I can use it again later.

4. As a student, I want clear READ prompts, WRITE output, and error messages so I can understand and troubleshoot program behavior.

5. As a student, I want to run the same program again without reopening it and adjust the GUI colors through configuration.

Architecture

Milestone 4 separates presentation, business logic, and data access so that the simulator does not depend on where a program was loaded from.

Tier	Component	Responsibility
Presentation	`main.py`, `gui/app.py`	Start the GUI, arrange controls, show the selected filename and status, and coordinate the editor, file actions, and execution controls.
Presentation	`gui/editor_panel.py` (`EditorPanel`)	Display numbered BasicML entries; support add, modify, delete, cut, copy, and paste; enforce the 100-entry limit; validate edits; track unsaved changes; and disable editing during execution.
Presentation	`gui/file_actions.py` (`FileActions`)	Handle Open, Save, and Save As dialogs, selected path, and confirmation before discarding unsaved changes.

Tier	Component	Responsibility
Presentation	`gui/io_panel.py`	Display WRITE output and handle GUI READ input.
Presentation	`gui/theme.py`	Apply configured colors to GUI widgets.
Business	`controller.py` (`SimulatorController`)	Manage execution state and coordinate the GUI with the simulator; accept program words rather than loading files directly.
Business	`uvsim.py` (`UVSim`)	Maintain machine state, fetch and execute instructions, and manage the instruction counter and accumulator.
Business	`word.py` (`Word`)	Validate signed four-digit words and expose the word value, opcode, operand, and formatted representation.
Business	`operations/` and `input_validation.py`	Implement the BasicML instructions and validate runtime input.
Data	`program_store.py` (`ProgramStore`)	Read and write BasicML program files, validating file contents and capacity without modifying simulator state.
Data	`theme_config.py`	Load the color configuration and provide UVU defaults when configuration is missing or invalid.

The program editor is the source of the program currently visible to the student. Opening a file places words in the editor; Run passes validated editor words to the controller; Save and Save As write those words through the program store. The controller and simulator do not need to know whether the words came from a file or from editing.

The `Word` class keeps word formatting and instruction decoding in one place. The data layer handles file I/O, the business layer handles execution, and Tkinter remains in the presentation layer. This is the team's Milestone 4 design. File commands operate on source; only Run initializes execution memory.

Machine state

State	Meaning and initial value
Memory	100 Word objects, addressed 00–99, initially zero. Each location may hold an instruction, data, or unused space.
Accumulator	Separate integer register for calculations, initially zero.
Instruction counter	Address of the next instruction, initially 00.
Instruction register	Most recently fetched Word, initially Word(0).
Halted flag	Whether HALT has stopped execution, initially false.

Words range from -9999 through 9999. File instructions use a plus sign; data may use either sign. An instruction's first two digits are its opcode and its last two digits are its operand address. For example, `+2007` loads memory address 07 into the accumulator. Instructions and data share memory; STORE and READ can replace words that were originally instructions.

Loading, editing, saving, and execution

1. The student selects **Open**. `FileActions` uses a file dialog to choose a BasicML file from any accessible folder.

2. `ProgramStore.load_source(path)` reads nonblank entries, retaining malformed words for repair. Read/encoding errors or more than 100 entries preserve existing source.

3. After Save/Discard/Cancel confirmation, `EditorPanel.set_entries(entries)` displays source at 00 onward, highlights invalid rows, and records a clean baseline. Opening leaves execution memory and prior output unchanged.

4. The student can add, modify, delete, cut, copy, or paste words. The editor tracks changes, reports invalid words, and rejects additions or pastes that would exceed 100 entries.

5. The student selects **Save** or **Save As**. The editor returns validated words through `get_words()`, and `ProgramStore.save(path, words)` writes one formatted word per line. A successful save clears the dirty flag.

6. The student selects **Run**. The application validates the current editor contents and passes the resulting `Word` objects to `SimulatorController.load_program(words)`, which resets the machine before execution.

7. The simulator fetches, decodes, and executes BasicML instructions. READ requests input through the GUI; WRITE sends values to the output panel. HALT ends execution. Stop or a runtime error also ends the current run.

8. While the program runs or waits for READ input, the editor is read-only. After execution ends, the student can edit and run the same program again without reopening its file.

9. At startup, the GUI loads colors from a configuration file. Missing or invalid configuration falls back to the UVU defaults.

The 100 memory addresses remain 00–99. File contents may include data words, so GUI loading enforces capacity while retaining malformed words for correction. Run and Save validate word format; neither requires every word to be an executable opcode. Instruction validity is checked when execution reaches it.

Use cases

The primary actor for every use case is the student running a BasicML program.

For UC-03 through UC-13, the shared precondition is that a valid program has been loaded into the simulator, execution has reached the relevant instruction, and the operand address is in 00–99. Unless specified otherwise, a successful instruction continues to the next address. Runtime errors stop the current execution and are reported in the GUI.

The identifiers below are retained for references from tests and the unit-test

spreadsheet.

UC-01 — Open and load a BasicML program

• **Precondition:** The student has launched the UVSim GUI.

• **Trigger:** The student selects Open and chooses a BasicML file.

Main flow:

1. The student selects a program file using the file dialog.

2. Read nonblank source entries and enforce the 100-entry limit.

3. Resolve unsaved changes through UC-25.

4. Display source entries at addresses starting at 00, highlighting invalid words.

5. Review or correct entries before execution.

• **Alternate/error flow:** Cancel, unreadable/undecodable files, and oversized files preserve source/path. Malformed BasicML words load for correction. Blank lines are ignored; GUI validation lines refer to editor rows, not original blank file lines.

• **Postcondition:** Source is available for editing; only valid nonempty source runs. Opening does not clear previous output.

UC-02 — Validate BasicML words and program capacity

• **Precondition:** The student has opened a program or is editing instructions.

• **Trigger:** A program is loaded, modified, or prepared for execution.

Main flow:

1. Validate each word using the signed four-digit BasicML format.

2. Accept values between -9999 and 9999.

3. Ensure the program contains no more than 100 entries.

4. Report the line or address of an invalid entry.

• **Alternate/error flow:** Invalid formatting or more than 100 entries produces an error. Rejected additions or pastes do not modify the program.

• **Postcondition:** Invalid source remains editable; Run/Save are blocked until words are valid. Run also rejects empty source; Save permits an empty file.

UC-03 — READ GUI input into memory (10)

• **Trigger:** Execution reaches READ.

Main flow:

1. Display a READ prompt in the GUI, accept the submitted integer, validate its range, and store it at the operand address.

• **Alternate/error flow:** Nonnumeric, blank, or out-of-range input is

reported so the student can correct it without changing the destination. READ accepts integers such as `7`, `-5`, or `+0007`; the fixed-width file format is not required for interactive input.

• **Postcondition:** The destination contains the accepted input; the accumulator

is unchanged.

UC-04 — WRITE memory to GUI output (11)

• **Trigger:** Execution reaches WRITE.

Main flow:

1. Display the operand memory word in the GUI output area using four zero-padded magnitude digits and a minus sign for negative values.

• **Alternate flow:** Negative values retain their minus sign. For example,

12 prints as `0012`, -5 as `-0005`, and zero as `0000`.

• **Postcondition:** One value is printed; memory and accumulator are unchanged.

UC-05 — LOAD accumulator (20)

• **Trigger:** Execution reaches LOAD.

Main flow:

1. Copy the operand memory word into the accumulator.

• **Alternate flow:** Zero and negative values are copied without changing their sign.

• **Postcondition:** The accumulator holds the selected word; memory is unchanged.

UC-06 — STORE accumulator (21)

• **Trigger:** Execution reaches STORE.

Main flow:

1. Copy the accumulator into the operand memory address, replacing its previous word.

• **Alternate flow:** The destination may previously have held data, an instruction,

or an unused zero word.

• **Postcondition:** The destination equals the accumulator; the accumulator is unchanged.

UC-07 — ADD (30)

• **Trigger:** Execution reaches ADD.

Main flow:

1. Add the operand memory word to the accumulator and retain the result in the accumulator.

• **Alternate/error flow:** Zero and negative operands are valid. A result outside

-9999 through 9999 discards higher-order magnitude digits, retains the sign, and continues.

• **Postcondition:** The accumulator contains the valid sum; memory is unchanged.

UC-08 — SUBTRACT (31)

• **Trigger:** Execution reaches SUBTRACT.

Main flow:

1. Subtract the operand memory word from the accumulator and retain the result in the accumulator.

• **Alternate/error flow:** Negative operands and negative results are valid.

A result outside the word range is truncated to four magnitude digits, retaining the sign.

• **Postcondition:** The accumulator contains the valid difference; memory is unchanged.

UC-09 — DIVIDE (32)

• **Trigger:** Execution reaches DIVIDE.

Main flow:

1. Divide the accumulator by the operand memory word.

2. Truncate any fractional part toward zero and retain the integer quotient in the accumulator.

• **Alternate/error flow:** For example, -7 divided by 3 yields -2. A zero divisor

raises `ValueError` and preserves the accumulator. The integer result is stored using the same sign-preserving four-digit truncation policy.

• **Postcondition:** The accumulator contains the valid quotient; memory is unchanged.

UC-10 — MULTIPLY (33)

• **Trigger:** Execution reaches MULTIPLY.

Main flow:

1. Multiply the accumulator by the operand memory word and retain the result in the accumulator.

• **Alternate/error flow:** A zero operand produces zero. Negative operands are

valid. A result outside the word range discards higher-order magnitude digits and retains the sign; execution continues.

• **Postcondition:** The accumulator contains the valid product; memory is unchanged.

UC-11 — BRANCH (40)

• **Trigger:** Execution reaches BRANCH.

Main flow:

1. Replace the instruction counter with the operand address, regardless of the accumulator's value.

• **Alternate flow:** The target can be earlier, later, or the current instruction.

Repeated branches can create a loop.

• **Postcondition:** The next instruction is fetched at the target address;

memory and accumulator are unchanged.

UC-12 — BRANCHNEG (41)

• **Trigger:** Execution reaches BRANCHNEG.

Main flow:

1. If the accumulator is negative, replace the instruction counter with the operand address.

• **Alternate flow:** If the accumulator is zero or positive, keep the sequential

next address already set by `step()`.

• **Postcondition:** Execution continues at the selected address; memory and

accumulator are unchanged.

UC-13 — BRANCHZERO (42)

• **Trigger:** Execution reaches BRANCHZERO.

Main flow:

1. If the accumulator equals zero, replace the instruction counter with the operand address.

• **Alternate flow:** If the accumulator is positive or negative, keep the

sequential next address already set by `step()`.

• **Postcondition:** Execution continues at the selected address; memory and

accumulator are unchanged.

UC-14 — Execute and halt a BasicML program (43)

• **Precondition:** A valid program is available in the editor.

• **Trigger:** The student selects Run.

Main flow:

1. The application retrieves and validates the current editor contents.

2. The simulator initializes memory and resets execution state.

3. Instructions execute until HALT, Stop, or an error occurs.

4. Output and status information are displayed in the GUI.

5. After execution ends, the student can run the program again without reopening the file.

• **Alternate/error flow:** An invalid instruction, out-of-range counter, or runtime error stops execution and displays an error. A program without HALT may loop until stopped.

• **Postcondition:** Execution stops while the program remains available for editing or another run.

UC-15 — View a program in the editor

• **Precondition:** A BasicML program has been loaded.

• **Trigger:** The student opens or loads a program.

Main flow:

1. Display each program word in the editor.

2. Show its corresponding memory address, from 00 through 99.

3. Allow the student to review the program before execution.

• **Postcondition:** The student can see the program's instructions and data values.

UC-16 — Add a program entry

• **Precondition:** The editor is enabled and contains fewer than 100 entries.

• **Trigger:** The student selects Add.

Main flow:

1. Insert +0000 before the first selected row, or append if no row is selected.

2. Shift following entries down.

3. Update the displayed addresses.

4. Mark the program as modified.

• **Alternate/error flow:** If the program already contains 100 entries, reject the addition and explain the limit.

• **Postcondition:** The new entry is available for editing.

UC-17 — Modify a program entry

• **Precondition:** The editor is enabled.

• **Trigger:** The student changes an instruction or data value.

Main flow:

1. Select an entry and change its value.

2. Record the modification and mark the program as having unsaved changes.

3. Validate the changed value before returning the program for execution or saving.

• **Alternate/error flow:** An invalid word produces an error identifying the affected line.

• **Postcondition:** The valid modified program can be saved or executed.

UC-18 — Delete program entries

• **Precondition:** The editor is enabled and contains entries.

• **Trigger:** The student selects one or more entries and chooses Delete.

Main flow:

1. Remove the selected entries.
2. Shift remaining entries up.
3. Update addresses to remain consecutive.
4. Mark the program as modified.

• **Postcondition:** The selected entries are removed.

UC-19 — Cut, copy, and paste program entries

• **Precondition:** The editor contains entries and the relevant operation is available.

• **Trigger:** The student selects Cut, Copy, or Paste.

Main flow:

1. Select one or multiple entries.
2. Copy places selected words on the clipboard without removing them.
3. Cut copies selected words and removes them.
4. Paste inserts clipboard words at the selected position.
5. Update addresses and track unsaved changes when entries are modified.

• **Alternate/error flow:** Invalid clipboard contents or a paste exceeding 100 entries are rejected without changing the program.

• **Postcondition:** The requested operation is completed when valid.

UC-20 — Reject edits exceeding memory capacity

• **Precondition:** The editor contains program entries.

• **Trigger:** An Add or Paste operation would exceed 100 entries.

Main flow:

1. Calculate the resulting number of entries.
2. Detect that the result exceeds the 100-address limit.
3. Reject the operation and display an explanation.
4. Preserve the existing program.

• **Postcondition:** The program remains unchanged and contains no more than 100 entries.

UC-21 — Prevent editing during execution

• **Precondition:** The simulator is running or waiting for READ input.

• **Trigger:** The student attempts to modify the program.

Main flow:

1. Disable editor modifications during execution.

2. Prevent Add, Delete, Cut, Paste, and direct editing.

3. Keep Copy available for reviewing program contents.

4. Re-enable editing when the simulator returns to an appropriate inactive state.

• **Postcondition:** The program cannot be modified while it is executing.

UC-22 - Save the current program

• **Precondition:** Execution is inactive.

• **Trigger:** Save, or Yes during an unsaved-change decision.

Main flow:

1. Commit any active cell edit and validate editor entries.

2. Without a current path, perform UC-23; otherwise use that path.

3. Write formatted words to a temporary file beside the destination, then replace the destination after writing succeeds.

4. Mark source saved and display the saved status/path.

• **Alternate/error flow:** Invalid words or write/replacement failure preserve source, dirty state, and path. A failed replacement preserves the original file. Empty source can be saved. Execution memory is never the saved source.

• **Postcondition:** A successful save matches validated editor contents.

UC-23 - Save As to a chosen file

• **Precondition:** Execution is inactive.

• **Trigger:** Save As, or Save without a path.

Main flow:

1. Choose a filename and accessible directory in the native dialog.

2. Validate current words and save through ProgramStore.save.

3. Update Selected File Name and mark source saved only after success.

• **Alternate/error flow:** Cancel, invalid words, or failed write preserve path and dirty state.

• **Postcondition:** The chosen file contains the source; later Save uses that path.

UC-24 - Configure application colors

• **Precondition:** The user can edit theme.ini beside the application.

• **Trigger:** Restart after editing the file.

Main flow:

1. Set primary and off_color under [theme] to #RRGGBB values.

2. Save the configuration and restart UVSim.

3. Apply both colors without recompiling or reinstalling.

• **Alternate/error flow:** Missing/unreadable/malformed configuration, missing keys, or invalid colors restore both UVU defaults (#4C721D and #FFFFFF). Native dialogs keep OS styling; custom contrast is not enforced.

• **Postcondition:** The configured palette, or both defaults, is applied.

UC-25 - Resolve unsaved edits before Open or Close

• **Precondition:** Editor entries differ from their saved baseline.

• **Trigger:** Open another source or close the window.

Main flow:

1. Commit an active cell edit and detect dirty source.

2. Offer Yes (Save), No (Discard), or Cancel (Keep editing), defaulting to Cancel.

3. Yes performs UC-22; proceed only on success. No proceeds without saving. Cancel keeps the current editor/window.

4. Open rereads the chosen source after confirmation in case Save changed it. Close cancels the timer and destroys the window.

• **Alternate/error flow:** Failed or canceled saves prevent Open/Close. Initial read errors do not discard edits.

• **Postcondition:** Replacement/close follows an accepted decision, never a failed save.

UC-26 - Stop execution

• **Precondition:** Running or waiting for READ.

• **Trigger:** Stop.

Main flow:

1. Discard the pending READ and enter stopped state.

2. Keep output/source and enable inactive controls and editing.

3. A later Run follows UC-14 from fresh memory; Stop is not pause/resume.

• **Alternate flow:** Stop is disabled outside active execution.

• **Postcondition:** Execution stops without altering source or its dirty state.

Error handling and implementation policies

The simulator uses the following policies for Milestone 4:

• BasicML files contain one signed four-digit word per line. Blank lines and

surrounding whitespace are permitted when loading. Values must be within -9999 through 9999, and a program cannot exceed 100 words.

• Invalid editor entries are reported with the affected line or address.

Additions or pastes exceeding 100 entries are rejected without modifying the program. Unsaved changes are tracked for confirmation before discard.

• File-open and save errors are reported in the GUI. An unsuccessful open

preserves the current editor contents; a failed save must not overwrite the destination with a partially written program.

• During execution or while waiting for READ input, editing is disabled.

Invalid READ input can be corrected without restarting the program.

• Arithmetic overflow discards higher-order magnitude digits while preserving

the sign (12345 becomes 2345; -12345 becomes -2345). Division truncates toward zero; division by zero is a runtime error.

• The simulator checks instruction validity during execution, not merely when

opening a file. Unsupported or negative instructions and invalid addresses stop the current run and produce an error message.

• Running again starts from a reset machine state using the editor's current

words; it does not require reopening the file. HALT ends execution normally.

• A missing or invalid theme configuration falls back to the default primary

color `#4C721D` and off-color `#FFFFFF`.

Legacy console mode remains available and uses strict ProgramStore.load validation. GUI Open uses load_source and accepts malformed words. Run/Save require valid entries. Opening another source retains prior output until a valid Run starts.

Verification and traceability

The current UVSim-Milestone-4-Unit-Tests.xlsx preserves existing UT IDs and adds all current methods. UC-01 through UC-14 retain operation IDs; UC-15 through UC-21 cover editing; UC-22 through UC-26 cover saving, branding, unsaved decisions, and Stop.

Automated tests are stored in `tests/` and run with

`python -m unittest discover -s tests -v`.

`Test1.txt` exercises input, addition, storage, output, and HALT. `Test2.txt`

uses subtraction and BRANCHNEG to print the larger of two inputs. The additional

Multiply, Divide, Branch, and BranchZero sample programs exercise the remaining

opcodes; commands and expected outputs are documented in `programs/README.txt`.

Manual sample checks complement the unit tests and are not additional unit-test

methods.

Current submission references

See srs/SRS_Final_Document.md, class-definitions.md, UVSim-Milestone-4-Unit-Tests.xlsx, and verification.md. Actual automated outcomes are distinguished from outstanding manual checks.